YA CASI ESTÁ

Cuatro compañeros, una app en React y una entrega que ya no espera

Personajes: Persona 1, Persona 2, Persona 3 y Persona 4 | **Duración:** aproximadamente 4-5 minutos

1. La última prueba

PERSONA 1 Ya, revisemos la página para inscribirse a la jornada de limpieza. Mañana la presentamos. Antes de que alguien escriba “ya casi está” al grupo, definamos qué significa casi, porque esa frase ya nos ha costado dos noches.

PERSONA 2 Se la enseñé a mi tía y no encontró el botón de inscripción. Le dije dónde estaba y me soltó: “¿Y si tú no estás, qué hago?”. Tiene razón. No puedo estar al lado de cada usuario haciendo de GPS humano.

PERSONA 4 Yo abrí una actividad y después no supe cómo volver a la lista. Usé la flecha del navegador. Funciona, pero no debería ser la solución oficial; uno entra a inscribirse, no a hacer turismo por la página.

PERSONA 3 Ordenemos la interfaz con componentes. La barra, las tarjetas y el formulario pueden ser piezas separadas. Cada componente cumple una función y lo podemos reutilizar, en vez de copiar el mismo bloque por todo el proyecto.

PERSONA 1 Me parece. El archivo está tan mezclado que cambié una frase y guardé otra versión por seguridad. En este grupo, “es un cambio chiquito” suele ser la forma elegante de anunciar trabajo extra.

2. Componentes y navegación

PERSONA 2 Todas las tarjetas usarán el mismo componente; solo cambia el título, la fecha y el lugar. Así el diseño queda parejo. Porque ayer alguien escribió la ubicación como “por la pollería de siempre”, y la pollería cerró hace dos años.

PERSONA 4 Aclárenme algo: ¿estamos usando componentes funcionales o de clase? Me acuerdo de ambos, pero no quiero decir una cosa y que luego el código me desmienta delante de todos.

PERSONA 3 Usamos funcionales, que se escriben como funciones y son los más comunes hoy. Pueden manejar estado con herramientas como useState. Los de clase siguen existiendo, sobre todo en proyectos antiguos. También podemos separar componentes visuales de otros que manejan lógica.

PERSONA 1 Listo. ¿Y cómo pasamos de Actividades al detalle y después a la inscripción sin hacer un botón nuevo para cada pantalla?

PERSONA 3 Con React Router podemos definir rutas como /actividades, /actividades/1 e /inscripcion y asociarlas con sus vistas. La navegación cambia la sección sin recargar toda la página, y la dirección ayuda a saber en qué parte estamos.

PERSONA 2 También hace falta una ruta para una dirección equivocada. Ayer me apareció una pantalla vacía y pensé que se había ido el internet. Acá uno reinicia el router primero y pregunta después.

PERSONA 4 Ya agregué un mensaje de “No encontramos esta página” y un enlace para volver a Actividades. Lo probé: ahora, si alguien se pierde, por lo menos la página le explica qué pasó.

3. El formulario tiene que avisar

PERSONA 1 En la inscripción pedimos nombre, correo y actividad elegida. Si falta un dato, hay que avisar antes de enviar; nadie debería descubrir el día de la jornada que su inscripción nunca se guardó.

PERSONA 4 Dejé el correo vacío y me dejó continuar. La página confía demasiado. Para recoger un paquete te piden confirmar hasta el apellido; aquí bastaba con pulsar el botón y encomendarse a la suerte.

PERSONA 2 Voy a mostrar el error debajo del campo que falta. Es más claro que lanzar una alerta que tapa todo y que uno cierra sin leer, igual que los términos y condiciones.

PERSONA 3 Hagamos un formulario controlado: React guarda los valores de los campos en el estado. La propiedad value muestra el valor actual y onChange actualiza el estado cuando se escribe. Al enviar, comprobamos campos vacíos y el formato del correo.

PERSONA 1 O sea, si borro el correo, React registra el cambio. Pero el aviso tiene que aparecer en el momento adecuado; tampoco vamos a recibir al usuario con un error antes de que haya empezado a escribir.

PERSONA 3 Exacto. Podemos validar al escribir o al enviar, según la experiencia que queramos. Y si los datos llegan a un servidor, también se validan allí: revisar la interfaz no reemplaza la validación del servidor.

PERSONA 4 Ahora sí me dice qué falta y mantiene los otros datos. Aprobado. Nadie quiere volver a llenar todo, como cuando una página pierde la sesión justo después de escribir media biografía.

4. Cuando el formulario crece

PERSONA 2 Por ahora hay pocos campos, pero después quieren añadir teléfono, disponibilidad, turnos y aceptación de condiciones. Con más reglas y mensajes, mantener todo manualmente puede volverse pesado.

PERSONA 4 ¿Ahí nos serviría React Hook Form o Formik? He escuchado los nombres, pero ¿qué cambia frente a manejar el formulario nosotros mismos?

PERSONA 3 Ambas bibliotecas ayudan con valores, errores, validación y envío. React Hook Form suele requerir menos código y busca reducir renderizados innecesarios. Formik también organiza los valores y errores. La elección depende de la complejidad y de lo que el equipo maneje mejor.

PERSONA 1 Entonces no instalamos una biblioteca solo porque está de moda. Para este formulario basta lo que tenemos; si crece, comparamos opciones. No alquilemos un local para guardar una bicicleta.

PERSONA 2 Yo termino el diseño y dejo los botones con nombres claros. Nada de poner “Continuar” en todas partes y esperar que la persona adivine si va a guardar, volver o aceptar su destino.

PERSONA 4 Voy a repetir el recorrido completo: actividades, detalle, inscripción y mis inscripciones. También probaré datos incorrectos. Esta vez seguiré el menú, no mi instinto ni la flecha del navegador.

PERSONA 3 Yo revisaré que rutas, componentes y formulario estén conectados. Si todo pasa, hacemos una última prueba juntos y entregamos.

PERSONA 1 Listo. Esta vez sí podremos escribir “ya casi está” con pruebas que lo respalden. Y si alguien propone otro cambio chiquito a medianoche, que lo guarde para la siguiente versión y duerma con su idea.

Fin